

Képfeldolgozás – EAN-13 vonalkód feldolgozás

Makai Marcell - D53G1U

Tartalomjegyzék

1	BEVEZETÉS.....	3
2	A FELADAT ÉS A PROBLÉMA MEGHATÁROZÁSA.....	3
3	AZ EAN-13 KÓDRENDSZER ÁTTEKINTÉSE	3
3.1	AZ EAN-13 SZEREPE A TERMÉKAZONOSÍTÁSBAN	4
3.2	A 13 SZÁMJEGY FELÉPÍTÉSE	4
3.3	A 95 MODULBÓL ÁLLÓ VONALKÓDSZERKEZET	5
3.4	L, G, ÉS R KÓDOLÁSI MINTÁK	5
3.5	ELLENŐRZŐ SZÁMJEGY ÉS HIBAFELISMERÉS	5
3.6	QUIET ZONE ÉS A VONALKÓD KÖRNYEZETE	6
4	A PROGRAM FŐ MODULJAI	6
4.1	MAIN.PY	6
4.2	BARCODE_DETECTION.PY	6
4.3	IMAGE_PROCESSING.PY	6
4.4	SCANLINE.PY	7
4.5	PATTERN_DECODER.PY	7
4.6	DECODER.PY	7
4.7	EAN13_PATTERNS.PY	7
4.8	BARCODE_READER.PY	7
4.9	STATS_REPORT.PY	7
5	FELDOLGOZÁSI FOLYAMAT	7
6	ELŐFELDOLGOZÁS ÉS ROI DETEKTÁLÁS.....	8
7	SCANLINE-ALAPÚ DEKÓDOLÁS.....	8
8	RUN-ALAPÚ ÉS BIT-ALAPÚ DEKÓDOLÁS.....	8
9	HIBAKEZELÉS ÉS BIZONYTALANSÁGI LOGIKA.....	9
10	STATISZTIKA ÉRTÉKELÉSE	9
10.1	HIBAKATEGÓRIÁK ELEMZÉSE	10
10.2	SCORE ÉS TALÁLATSZÁM ÖSSZEHASONLÍTÁSA	11
10.3	A HIBÁS OLVASÁSOK JELENTŐSÉGE	11
10.4	MÓDSZERCSALÁDOK SZEREPE.....	12
10.5	ROI DETEKTÁLÁS ÉRTÉKELÉSE	12
11	EREDMÉNYEK ÉRTELMEZÉSE	13
12	FEJLESZTÉSI LEHETŐSÉGEK	13
13	HASZNÁLATI LEÍRÁS	14
14	ÖSSZEGZÉS.....	15
	FORRÁSOK.....	16

1 Bevezetés

A projekt célja, egy Pythonban megvalósított, képfeldolgozáson alapuló EAN-13 vonalkódolvasó fejlesztése és tudásának értékelése [8]. Az alkalmazás feladata, hogy a termékfotókon automatikusan megtalálja a vonalkód-gyanús területeket, ezeket előfeldolgozza, majd scanline-alapú dekódolással megpróbálja kiolvasni az EAN-13 kódot belőle.

A legnagyobb kihívást az jelenti, hogy a vonalkód képek nem laborkörülmények között készültek, így gyakran ferdék, részben elmosódottak, eltérő megvilágításúak, csomagolásgörbületen helyezkednek el, vagy szöveges / táblázatos grafikai elemekkel keverednek. Ez miatt nem elég, ha dekódolni tudja az alkalmazás, hanem a bizonytalan vagy hibás olvasásokat is kezelnie kell.

A dokumentáció bemutatja az EAN-13 szabvány alapjait, az alkalmazás felépítését, az alkalmazott képfeldolgozási és dekódolási lépéseket, a hibakezelési logikát, valamint egy 1200 képből álló tesztadatbázison mért eredményeket. [1], [2], [7], [9]

2 A feladat és a probléma meghatározása

A megoldandó feladat egy olyan EAN-13 scanner elkészítése volt, amely képfájlokból képes automatikusan kiolvasni az azonosító számot. A bemeneti képek fájlneve tartalmazza az elvárt EAN-kódot, ezért a rendszer automatikusan össze tudja hasonlítani a dekódolt eredményt a várt értékekkel.

A feldolgozás eredménye három fő kategóriába sorolható:

- **OK:** a rendszer által kiolvasott EAN-13 kód megegyezik a fájlnevből származó elvárt értékkel.
- **WRONG / ROSSZ:** a rendszer adott vissza EAN-13 kódot, de az nem egyezik meg az elvárt értékkel.
- **ERROR / HIBA:** a rendszer nem adott vissza kódot, mert nem talált megfelelő ROI-t, nem tudta stabilan dekódolni a scanline-okat, vagy bizonytalannak minősítette az eredményt.

A fejlesztés során a legfontosabb cél nem pusztán az olvasási arány növelése volt, hanem a hibás, de checksum-helyes hamis pozitív találatok csökkentése. Egy valós alkalmazásban a rossz vonalkód visszaadása súlyosabb probléma, mint az, ha a rendszer jelzi hogy nem biztos az olvasásban.

3 Az EAN-13 kódrendszer áttekintése

Az EAN-13 egy 13 számjegyből álló termékazonosító kódrendszer, amely a GS1 szabványrendszer részeként a kereskedelmi termékazonosítás egyik elterjedt formája [1],

[2]. A rendszer célja, hogy a termékeket egyértelműen azonosíthatóak legyenek a kereskedelmi, logisztikai és készletkezelési folyamatokban. A vonalkód grafikus formában jeleníti meg a számsorozatot, így optikai eszközökkel vagy képfeldolgozó algoritmusokkal automatikusan beolvasható.

A projekt szempontjából azért fontos az EAN-13 felépítésének ismerete, mert a program nem egyszerű karakterfelismerést végez, hanem a vonalkód szabványos sávstruktúráját próbálja visszafejteni. A sikeres dekódoláshoz a rendszernek fel kell ismernie a kezdő, középső és záró őrjeleket, a számjegyekhez tartozó sáv szélesség mintákat, a bal oldali paritás mintát és az ellenőrző számjegyet is [2], [8].

3.1 Az EAN-13 szerepe a termékazonosításban

Az EAN-13 kód a termékek globális azonosítására szolgál. A számsor általában egy GS1 vállalati azonosítóból, termékazonosító részből és egy ellenőrző számjegyből áll. A pontosan belső felosztás a kiadott azonosító hosszától és a GS1 szervezeti kiosztástól függ, de a gyakorlati felhasználás szempontjából a teljes 13 számjegy együtt azonosítja a terméket [1], [3].

A vonalkódolvasó alkalmazás számára ez azt jelenti, hogy a cél nem csupán egy tetszőleges 13 jegyű szám felismerése, hanem egy olyan számsor visszaadása, amely megfelel az EAN-13 formai és matematikai szabályainak. A dekódolás végén éppen ezért szükséges az ellenőrző számjegy vizsgálata. Ha a checksum nem egyezik, akkor a kapott kiolvasott számsor biztosan hibásnak tekinthető [2], [3].

3.2 A 13 számjegy felépítése

Az EAN-13 számsor 13 decimális számjegyből áll. Az első 12 számjegy hordozza az azonosító tényleges adattartalmát, a 13. számjegy az ellenőrző számjegy. Az ellenőrző számjegy számítása súlyozott összeggel történik: az egyes pozíciók értékei felváltva 1-es és 3-as súlyt kapnak, majd az eredmény alapján meghatározható az a számjegy, amely a teljes kódot tízzel oszthatóvá teszi [2], [3].

A programban ez a logika azért fontos, mert a dekódolt számjegysor csak akkor fogadható el jelöltként, ha az utolsó számjegy megegyezik a számított checksum értékével [8]. Ez azonban csak egy alapvető hibaszűrés. A checksum képes sok elírás vagy dekódolási hiba kiszűrésére, de nem bizonyítja teljes biztonsággal, hogy a kód valóban a képen szereplő vonalkódból származik.

Ez gyakorlatban azért lényeges, mert zajos vagy rosszul kivágott képrészleten is előfordulhat, hogy a rendszer véletlenül olyan 13 számjegyű sorozatot talál, amely a checksum szempontból érvényes. Emiatt a projektben a checksum mellett további megbízhatósági feltételek is szerepelnek, például a score, a találati szám, a módszercsaládok közötti egyezés és az alternatív jelöletek vizsgálata [8], [9].

3.3 A 95 modulból álló vonalkódszerkezet

Az EAN-13 grafikus vonalkódja nem közvetlenül a számjegyek képként történő felismerésén alapul, hanem egy szabványos modulrendszeren. A teljes EAN-13 vonalkód 95 azonos szélességű modulból áll [2]. Ezek a modulok fekete vagy fehér értéket vehetnek fel, és együtt alkotják a látható sávokat.

A 95 modul szerkezete:

- start guard: 3 modul
- bal oldali 6 számjegy: 6 x 7 modul
- middle guard: 5 modul
- jobb oldali 6 számjegy: 6 x 7 modul
- end guard: 3 modul

Ez összesen $3 + 42 + 5 + 42 + 3 = 95$ modul. A guard minták segítenek a vonalkód elejének, közepének és végének felismerésében. A start és end guard mintája fekete-fehér-fekete szerkezetű, a középső guard pedig elválasztja a bal és jobb oldali számjegycsoportot.

3.4 L, G, és R kódolási minták

Az EAN-13 számjegyei 7 modul szélességű mintákkal vannak kódolva. A jobb oldali hat számjegy R mintákkal jelenik meg, a bal oldali hat számjegy pedig L vagy G mintákkal. Az L és G minták nem véletlenszerűek: a bal oldali hat számjegy L/G sorrendje határozza meg az EAN-13 kód első számjegyét [2].

Ez azt jelenti, hogy az első számjegy nem külön 7 modulból álló vonalcsoportként szerepel a vonalkódon. Ehelyett a dekóder a bal oldali hat számjegy paritásmintájából következteti ki. Például, ha a bal oldalon minden számjegy L típusú mintával szerepel, az egy adott első számjegyet jelöl, más L/G kombinációk pedig más-más első számjegyhez tartoznak [2].

3.5 Ellenőrző számjegy és hibafelismerés

Az ellenőrző számjegy az EAN-13 egyik legfontosabb hibafelismerési eszköze. A dekódolt első 12 számjegyből a program kiszámítja a 13. számjegyet, majd összehasonlítja azt a ténylegesen olvasott utolsó számjeggyel [2], [3], [8]. Ha a két érték eltér, akkor a kód biztosan nem fogadható el.

Az ellenőrző számjegy ugyanakkor nem azonos a teljes olvasási bizonyossággal. A checksum csak a számsor matematikai helyességét vizsgálja, de nem ellenőrzi a képi eredet megbízhatóságát. Emiatt előfordulhat, hogy egy hibásan felismert, de véletlenül checksum-helyes számsor átmenne ezen az ellenőrzésen.

Emiatt a fejlesztett alkalmazásban további ellenőrzéseket csinálunk. A végső döntés több scanline eredményének összesítésén alapul. Ha ugyanazt a kódot sok scanline, több ROI-

forogtatás és több binarizálási módszer is támogatja, akkor az eredmény megbízhatónak tekinthető. Ha viszont csak kevés scanline ad azonos eredményt, vagy több checksum-helyes jelölt hasonló pontszámot kap, akkor a rendszer inkább bizonytalannak jelöli az olvasást [8], [9].

3.6 Quiet zone és a vonalkód környezete

Az EAN típusú vonalkódok olvasásánál fontos szerepet játszik a quiet zone, vagyis a vonalkód két oldalán található üres, zavaró mintáktól mentes terület. Ez segíti az olvasót abban, hogy pontosan felismerje, hol kezdődik és hol végződik a vonalkód [1], [2].

Termékfotókon ez gyakran problémát okoz. A vonalkód mellett közvetlenül szöveg, tápértéktáblázat, csomagolás grafika vagy más fekete-fehér elem helyezkedhet el. Ilyenkor a scanner egyszerűen tévesztheti össze a vonalkód sávjait a környező grafikai elemekkel, vagy a scanline nem tisztán a vonalkódon halad át.

4 A program fő moduljai

A program főbb, logikailag elkülönített modulból áll, a bemutatott moduláris felépítés a saját megvalósításom felépítését mutatja be [8]. Ez a felépítés átláthatóbbá teszi a fejlesztést és a dokumentálást, mert a képfeldolgozás, a ROI keresés, a scanline előállítás, a dekódolás és a statisztikai értékelés külön fájlokban található.

4.1 main.py

A fő belépési pont. Feladata a képek bejárása, az elvárt EAN-kód fájlnevből való kinyerése, a scanner meghívása, majd az eredmények kiírása. A futás során a program OK, ROSSZ vagy HIBA státuszt jelenít meg.

4.2 barcode_detection.py

A vonalkód-gyanús területek kereséséért felel. A megoldás éldetektálást, morfológiai műveleteket, kontúrkeresést és forogtatott téglalap alapú kivágást használ, amelynek az OpenCV képfeldolgozási könyvtárra épülnek [4], [6]. A cél az, hogy a teljes képből olyan kisebb ROI-k készüljenek, amelyek nagy eséllyel tartalmazzák a vonalkódot.

4.3 image_processing.py

Az alapvető képfeldolgozási műveleteket tartalmazza: képbetöltés, szürkeárnyaltos konverzió, Otsu-küszöbölés és adaptív küszöbölés [5]. Ezek a lépések a változó fényviszonyok miatt szükségesek.

4.4 scanline.py

A binarizált képből vízszintes scanline-okat állít elő, majd ezekből fekete-fehér run sorozatokat készít. A run-length reprezentáció azért hasznos, mert az EAN-13 számjegyek valójában fekete és fehér sávok relatív szélességeiből állnak.

4.5 pattern_decoder.py

A run-length alapú EAN-13 dekóder a bal oldali számjegyeket L/G, a jobb oldali számjegyeket R mintákkal hasonlítja össze. A dekódolás során távolságot számít a mért run-arányok és az ismert EAN-13 minták között.

4.6 decoder.py

A klasszikus 95 bites EAN-13 bitstruktúra dekódolásáért és a checksum ellenőrzésért felel [2]. Ez a modul tartalmazza az ellenőrző számjegy számítását is.

4.7 ean13_patterns.py

Az EAN-13 L, G és R bitmintáit, valamint az első számjegyhez tartozó paritásmintákat tartalmazza [2].

4.8 barcode_reader.py

A teljes olvasási folyamatot koordináló modul. Meghívja a ROI detektort, előállítja a bináris variánsokat, scanline-okat mintavételez, megpróbálja a run-alapú dekódolást, szükség esetén bit-alapú fallback dekódolást használ, majd több találat alapján kiválasztja vagy elutasítja a végső eredményt.

4.9 stats_report.py

A statisztikai kiértékelést végző modul [9]. Nem módosítja a feldolgozási logikát, hanem a meglévő olvasófüggvényt hívja meg, majd az eredményeket státusz, hibakategória, score, találatszám és módszerek szerint összesíti.

5 Feldolgozási folyamat

A feldolgozás fő lépései a saját implementáció alapján a következők [8]:

1. Kép betöltése
2. Kép több forgatási variánsának előállítása
3. Vonalkód-gyanús ROI-k keresése
4. ROI-k előfeldolgozása szürkeárnyaltos és bináris képpé
5. Több binarizálási módszer kipróbálása
6. Több scanline mintavételezése a ROI középső tartományából
7. Run-length sorozatok előállítása
8. Run-alapú EAN-13 dekódolás

9. Sikertelenség esetén bit-alapú fallback dekódolás
10. Checksum ellenőrzés
11. Több scanline-találat összesítése
12. Bizonytalansági szabályok alkalmazása
13. OK, ROSSZ, HIBA státusz megállapítása

6 Előfeldolgozás és ROI detektálás

A ROI detektálás célja az, hogy a teljes képből a vonalkódot tartalmazó részeket kivágja [8]. A program Sobel-alapú vízszintes gradienskeresést használ, mert a vonalkód sok, egymáshoz közeli függőleges sávból áll, amelyek erős intenzitásváltozást okoznak vízszintes irányban [4], [8].

A gradiens képen Gaussian blur csökkenti a zajt, majd Otsu-küszöbölés választja szét az erősebb éleket a háttértől [5], [8]. Ezután morfológiai zárás kapcsolja össze a vonalkód egymáshoz közeli sávjait egy nagyobb blokkszerű területté [6], [8]. A kontúrokból forgatott téglalapok készülnek, majd a program méret, oldalarány és terület alapján választja ki a legerősebb jelölteket.

7 Scanline-alapú dekódolás

A program nem egyetlen sort próbál olvasni, hanem sok scanline-t mintavételez a ROI középső tartományából [8]. Ennek az oka, hogy egy termékfotón a vonalkód gyakran ferde, görbült, részben homályos vagy lokálisan sérült. Több scanline használatával nő az esély arra, hogy legalább néhány sor stabilan átmenjen a vonalkód teljes szélességén.

A scanline-okból run-length sorozat készül. Egy run azt jelenti, hogy egymás után hány azonos színű pixel következik. Az EAN-13 számjegyei sávszélesség-arányokkal írhatóak le, ezért a run-alapú dekódolás közvetlenül illeszkedik a vonalkód természetéhez.

8 Run-alapú és bit-alapú dekódolás

Az elsődleges dekódolási módszer a run-alapú mintaillesztés [8]. A program a mért run-arányokat összehasonlítja az EAN-13 ismert L, G és R számjegymintáival. Minden számjegyre egy távolságérték tartozik, minél kisebb ez az érték, annál jobban hasonlít a mért minta az adott számjegyre.

Ha a run-alapú módszer sikertelen, a program fallback módszerként megpróbálja a run sorozatot 95 bites EAN struktúrává alakítani [2], [8]. Ezután ellenőrzi a start, middle és end guard biteket, majd a 7 bites számjegycsoportokat dekódolja. Ez a fallback bizonyos esetekben segít, de általában kevésbé stabil, mint a run-alapú illesztés.

9 Hibakezelés és bizonytalansági logika

A rendszer nem minden checksum-helyes kódot fogad el automatikusan [8]. Ez fontos döntés, mert a véletlenül talált hamis minták is eredményezhetnek checksum-helyes EAN-13 számot. A program ezért score, találatszám, módszercsalád és alternatív jelöltek alapján dönt.

A bizonytalansági logika célja, hogy a scanner inkább hibát jelezzon, mintsem rossz kódot adjon vissza. Ez különösen fontos olyan környezetben, ahol a visszaadott vonalkód alapján termékazonosítás, készletkezelés vagy további automatikus feldolgozás történik.

10 Statisztika értékelése

A rendszer kiértékelése 1200 darab termékfotón történt, a DEAL KAIST Lab Barcode Dataset felhasználásával [7], [9]. A tesztelés során minden képhez ismert volt az elvárt EAN-13 kód, mivel ez a fájlnevből kinyerhető volt. Ez tette lehetővé, hogy automatikusan össze lehessen hasonlítani a kapott eredményt a valós kóddal, ez alapján pedig a rendszer a három kategória egyikébe sorolta az olvasást: helyes olvasás, hibás olvasás vagy sikertelen olvasás.

A mérés eredményei alapján a rendszer 713 képet olvasott be helyesen, 477 esetben nem adott vissza kódot, és 10 esetben adott vissza hibás kódot [9]. Ez százalékosan 59,42% helyes olvasási arányt, 39,75% sikertelen olvasási arányt és 0,83% hibás olvasási arányt jelent.

Kategória	Darabszám	Arány
Helyes olvasás (OK)	713	59,42%
Sikertelen olvasás (HIBA)	477	39,75%
Hibás olvasás (ROSSZ)	10	0,83%
Összesen	1200	100%

Az eredmények értelmezésénél fontos különbséget tenni a sikertelen és a hibás olvasás között. A sikertelen olvasás azt jelenti, hogy a rendszer nem volt elég biztos az eredményben, ezért nem adott vissza kódot. A hibás olvasás ezzel szemben azt jelenti, hogy a rendszer adott vissza egy checksum-helyes EAN-13 kódot, de az eltért az elvárt értéktől. Gyakorlati felhasználásban a hibás olvasás veszélyesebb, mert a rendszer ilyenkor látszólag sikeresen működik, mégis rossz terméket azonosít.

A jelenlegi eredmények alapján a rendszer inkább konzervatív működésű: sok bizonytalan esetet elutasít, viszont alacsonyan tartja a hibás pozitív találatok számát. Ez a vonalkódolvasás szempontjából kedvező kompromisszum, mert a téves azonosítás általában nagyobb kockázatot jelent, mint az, ha a felhasználónak újra kell fotóznia vagy közelebbről kell beolvasnia a terméket.

10.1 Hibakategóriák elemzése

A 477 sikertelen olvasás három fő kategóriába sorolható [9]. A leggyakoribb hibatípus a *tul_keves_valtas*, amely 258 alkalommal fordult elő, és az összes hiba 54,09%-át adta. Ez azt jelenti, hogy a vizsgált scanline-okon nem jelent meg elég fekete-fehér váltás ahhoz, hogy a szabályos EAN-13 mintázatot lehessen felismerni.

A második leggyakoribb kategória a *bizonytalan_olvasas*, amely 207 esetben jelent meg, vagyis a hibás 43,40%-át tette ki. Ebben az esetben a rendszer talált checksum-helyes jelölteket, de azok nem kaptak elég erős támogatást, vagy több eltérő jelölt egymáshoz túl közeli pontszámot kapott. A harmadik kategória a *nem_talalt_roi*, amely mindössze 12 esetben fordult elő, vagyis a hibák 2,52%-át jelentette.

Hibakategória	Darabszám	Arány a hibákon belül	Értelmezés
<i>tul_keves_valtas</i>	258	54,09%	A scanline nem tartalmaz elég stabil fekete-fehér átmenetet.
<i>bizonytalan_olvasas</i>	207	43,40%	Van jelölt, de nem elég domináns vagy nem kap elég támogatást.
<i>nem_talalt_roi</i>	12	2,52%	A rendszer nem talált megfelelő vonalkód-gyanús területet.

Ez az eloszlás azt mutatja, hogy a rendszer fő problémája már nem elsősorban a vonalkód helyének megtalálása. A *nem_talalt_roi* aránya alacsony, tehát a ROI detektálás a legtöbb képen legalább valamilyen használható jelöltet előállít. A hibák nagyobb része inkább a ROI-n belüli dekódolási stabilitáshoz kapcsolódik: a kivágott részben a vonalkód vagy túl ferde, túl elmosódott, túl kis felbontású, vagy a scanline nem a sávokra merőlegesen halad át.

A *tul_keves_valtas* kategória különösen fontos, mert ez gyakran arra utal, hogy a vizsgált sor nem a tényleges vonalkódon halad keresztül, hanem háttéren, szövegen, csomagoláselemeken, vagy csak a vonalkód egy részén. Ez előfordulhat akkor is, ha a ROI tartalmazza a vonalkódot, de túl nagy területet fog át, és a scanline-ok jelentős része nem a hasznos sávokon fut.

A *bizonytalan_olvasas* kategória már egy fejlettebb hibakezelési állapot. Ilyenkor a rendszer nem teljesen sikertelen, mert talál lehetséges EAN-13 jelölteket. Azonban a score, a találatszám vagy az alternatív jelöltek közelsége alapján nem tekinti biztonságosnak az eredményt. Ez a kategória részben a hamis pozitívok elleni védelem következménye, és tudatosan vállalt kompromisszum.

10.2 Score és találatszám összehasonlítása

A statisztika alapján a helyes és hibás olvasások közötti jelentős különbség látható a score és a találatszám tekintetében [9]. A helyes olvasások átlagos score értéke 757,33, mediánja 503,08 volt. Ezzel szemben a hibás olvasások átlagos score értéke csak 26,60, mediánja pedig 18,28 volt.

Mutató	Helyes olvasások	Hibás olvasások
Átlagos score	757,33	26,60
Medián score	503,08	18,28
Átlagos találatszám	935,91	33,90
Medián találatszám	645,00	24,00

Ez az eltérés arra utal, hogy a score és a találatszám jól használható megbízhatósági mutató. A helyes olvasásoknál jellemzően sok scanline ugyanazt a kódot támogatja, míg a hibás olvasásoknál kevés sorból és alacsony pontszámból áll össze a találat.

A medián értékek különösen beszédesek, a helyes olvasások medián találatszáma 654, míg a hibás olvasásoké 24. Ez azt jelenti, hogy egy tipikus helyes beolvasást több száz scanline támaszt alá, míg egy tipikus hibás olvasást csak néhány tucat. Ez alapján további biztonsági szabályként alkalmazható lenne egy minimális találatszám vagy score-határ, különösen azoknál az eseteknél, ahol csak egy módszer család támogatja a jelöltet.

A score-ok közötti különbség ugyanakkor nem jelenti azt, hogy minden alacsony score hibás lenne. A tesztben előfordultak alacsony score-ú, de helyes olvasások is. Ezért a score önmagában nem elegendő döntési feltétel. Megbízhatóbb eredményt ad, ha a rendszer egyszerre veszi figyelembe a score-t, a találatszámot, a módszerek sokféleségét, valamint az alternatív jelöltek pontszámát.

10.3 A hibás olvasások jelentősége

A 10 hibás olvasás külön elemzést igényel, mert ezek képviselik a rendszer legkritikusabb hibáit [9]. A hibás olvasások közös jellemzője, hogy a program olyan mintázatot talált, amely megfelelt az EAN-13 checksum szabályának, de nem a képen szereplő tényleges vonalkódot adta vissza.

A statisztikában több hibás esetről látható, hogy a találatot viszonylag kevés scanline támogatta. Például egyes hibás olvasásoknál 12, 14, 24 vagy 32 találat szerepel, miközben a helyes olvasásoknál a tipikus találatszám több száz. Ez arra utal, hogy ezek a hibák gyakran lokális, véletlenszerű mintázatokból, szövegből, táblázatból, grafikai elemből vagy részleges vonalkód-szakaszból keletkeznek.

Külön figyelmet érdemelnek azok az esetek, ahol a hibás olvasás magasabb score-t kapott, például 40-60 körüli értéket. Ezek nem egyszerű zajból származó hibák, hanem olyan képrészletek, amelyek több scanline-on is hasonló, de téves mintát

eredményeztek. Ilyen esetben a további fejlesztésnek nemcsak score küszöböt kell növelnie, hanem a térbeli konzisztenciát is vizsgálnia kell, azaz valóban ugyanabból a vonalkódterületből származnak-e a találatok, szabályos-e a sávok geometriája, és megfelelő-e a quiet zone a vonalkód két oldalán.

A hibás olvasások aránya alacsony, mindössze 0,83%-os eredménye kedvező, de dokumentáció szempontjából fontos megemlíteni, hogy ez a legkritikusabb hibacsoport. Egy automatikus rendszerben egy sikertelen olvasás könnyen kezelhető újrapróbálással, míg egy hibás olvasás téves adatbevitelhez vezethet.

10.4 Módszercsaládok szerepe

A statisztika külön gyűjti, hogy mely binarizálási és dekódolási módszerek járultak hozzá a találatokhoz [9]. A legtöbb találat az Otsu küszöböléshez és a run-alapú dekódoláshoz kapcsolódott. Különösen erős volt a *roi0/otsu* és a *roi180/otsu* módszercsalád szerepe.

Ez arra utal, hogy a tesztképek jelentős részén a globális küszöbölés elegendő volt a vonalkód sávjainak elkülönítésére. Az adaptív küszöbölés szintén fontos szerepet játszott, főleg a változó megvilágítású vagy árnyékos képeknél. Az invertált módszerek viszont jóval kisebb arányban járultak hozzá a találatokhoz, és a hibás olvasások között is megjelentek. Emiatt az invertált találatok szigorúbb kezelése indokolt.

A módszercsaládok elemzése azért hasznos, mert nemcsak azt mutatja meg, hogy sikerüle-e olvasni, hanem azt is, milyen típusú előfeldolgozás bizonyult hatékonynak. Ha egy jelöltet több különböző módszer, több ROI-forgatás és több scanline is támogat, akkor az eredmény megbízhatóbb. Ha viszont csak egyetlen módszercsalád ad találatot, különösen alacsony score mellett, akkor nagyobb a hamis pozitív kockázata.

10.5 ROI detektálás értékelése

A *nem_talalt_roi* kategória mindössze 12 esetben jelent meg [9]. Ez alapján a ROI detektálás a teljes hibamennyiséghez képest viszonylag ritkán bukik el teljesen. Ez pozitív eredmény, mert a korábbi fejlesztési fázisokban a vonalkódterület megtalálása még jelentős problémát okozott, különösen oldalra fordított, csomagolásba ágyazott vagy más grafikai elemekkel körülvett vonalkódok esetén.

Ugyanakkor az alacsony *nem_talalt_roi* arány jelenti azt, hogy a ROI-k minden esetben pontosan. A *tul_keves_valtas* hibák egy része valószínűleg olyan ROI-kból származnak, amelyek tartalmazzák ugyan a vonalkódot, de túl nagyok, túl ferdék vagy nem megfelelően vannak igazítva.

A jelenlegi rendszer tehát többnyire megtalálja a vonalkód környezetét, de nem mindig képes azt olyan normalizált formára hozni, amelyben a scanline-ok stabilan dekódolhatóak.

11 Eredmények értelmezése

Az eredmények alapján a rendszer jelenlegi állapota már akár használható kísérleti EAN-13 olvasóként, de nem tekinthető teljesen robosztus, ipari pontosságú megoldásnak [9]. A helyes olvasási arány közel 60%, ami nem ideális, ugyanakkor figyelembe kell venni, hogy a bemeneti képek nem kontrollált körülmények között készültek. A rendszer nem előre kivágott, jó minőségű vonalkódképeket kapott, hanem teljes termékfotókból kellett megtalálnia és értelmeznie a vonalkódot.

A legfontosabb pozitív eredmény a hibás olvasások alacsony aránya. A 0,83%-os ROSSZ kategória azt mutatja, hogy a bizonytalansági szabályok nagyrészt jól működnek. A rendszer sok esetben inkább nem ad vissza eredményt, ha az olvasás nem elég megbízható. Ez különösen fontos olyan alkalmazási környezetben, ahol a visszaadott kód alapján automatikus termékazonosítás történik.

A HIBA kategória magas aránya ugyanakkor azt jelzi, hogy a rendszer érzékeny a képminőségre, a perspektívára és a vonalkód geometriájára. A legtöbb hibva nem abból fakad, hogy a program egyáltalán nem talál vonalkódot, hanem abból, hogy a kivágott területből nem nyerhető ki elég stabil EAN-13 mintázat. Ez alapján a további fejlesztéseknek elsősorban nem a szabványos dekóderlogikára, hanem a képi normalizálásra és a scanline-választásra kell koncentrálniuk.

A mérés azt is megmutatja, hogy a rendszerben alkalmazott többpróbás stratégia hasznos. A különböző forgatások, küszöbölési módszerek, scanline-magasságok kombinációja sok esetben erős konszenzust eredményez. A helyes találatokra jellemző, hogy több módszer és sok scanline ugyanazt az EAN-kódot erősíti meg. Ez a konszenzus-alapú működés a projekt egyik legfontosabb megbízhatósági eleme.

Összességében az eredmények alapján a rendszer fejlesztési irány egyértelműen meghatározható. A hibás pozitívok további csökkentése mellett a legnagyobb javulás a sikertelen olvasások csökkentéséből várható. Ehhez a vonalkód pontosabb geometriai illesztése, a zajos scanline-ok kiszűrése és a perspektíva vagy görbületi kompenzálás szükséges.

12 Fejlesztési lehetőségek

A statisztikák alapján az alábbi fejlesztési irányok javasolhatóak:

1. **Perspektíva- és görbekorrekció javítása:** a csomagolás görbülete miatt a vonalak nem mindig párhuzamosak, ezért jelenlegi téglalap-alapú kivágás nem minden esetben elegendő.
2. **ROI-n belüli vonalkód-tengely pontosítása:** a kivágott ROI-n belül külön meg lehetne keresni a tényleges vonalkód sávjainak irányát és középvonalát.

3. **Dinamikusabb scanline-választás:** a jelenlegi egyenletes mintavételezés helyett előnybe lehetne részesíteni azokat a sorokat, ahol sok szabályos fekete-fehér váltás található.
4. **Hamis pozitívok további szűrése:** az invertált, vagy csak egy módszercsalád által támogatott találatoknál még szigorúbb szabályok alkalmazhatóak.
5. **Kézi vagy automatikus finom ROI vágás:** dokumentációs és tesztelési célra hasznos lenne összehasonlítani a teljes képes és kézzel kivágott vonalkódos eredményeket.
6. **Külső barcode könyvtárral való összehasonlítás:** a saját fejlesztésű rendszer eredményei összevetése egy ipari, nyílt forrású olvasóval.

13 Használati leírás

A program parancssorból futtatható Python alkalmazásként használható. A futtatás előtt aktiválni kell a virtuális környezetet, majd a *main.py* fájlra kell átadni egy kép vagy egy teljes mappa helyét. Egyetlen kép feldolgozásakor a program az adott képre próbál EAN-13 kódot olvasni, mappa megadásakor pedig a mappában található képeket dolgozza fel egymás után [8].

Általános futtatás:

```
source .venv/bin/activate
python main.py [kép / mappa]
```

A statisztikai riport külön fájlal készíthető el. A *stat_report.py* nem módosítja a barcode-feldolgozást, hanem ugyanazt az olvasási függvényt használja, mint a főprogram, majd az eredményeket strukturált formában összegyűjti [9].

Statisztikai riport készítése:

```
python stats_report.py tests/images/rec
python stats_report.py tests/images/single_test
```

A riportgenerálás három kimeneti fájlt hoz létre:

```
scanner_report.md      # Markdown formátumú riport
scanner_result.csv     # Képenkénti részletes eredménylista
scanner_result.json    # Géppel könnyebben feldolgozható lista
```

A statisztikai riport az OK, ROSSZ és HIBA kategóriák darabszámát és arányát, a hibatípusok gyakoriságát, a score és találatszám statisztikáit, a leggyakoribb módszercsaládok, valamint a legfontosabb hibás beolvasásokat tartalmazza [9]. Ezen kimenet alapján készítettem el a kiértékelést.

14Összegzés

A fejlesztett EAN-13 vonalkódolvasó képes nem ideális termékfotókon ROI-kat keresni, többféle binarizálási módszert kipróbálni, több scanline alapján dekódolni, majd a találatokat statisztikailag értékelni [8], [9]. A rendszer nemcsak eredményt ad, hanem részletes hibamagyarázatokat és összesített statisztikát is készít.

Az 1200 képes teszt alapján a rendszer jelenlegi állapotában 59,42%-os helyes olvasási arányt és 0,83%-os hibás olvasási arányt ért el. A legnagyobb további fejlesztési lehetőség a HIBA kategória csökkentése lehet, különösen a túl kevés fekete-fehér váltás és bizonytalan olvasás kategóriákban.

Források

[1] GS1, „Barcodes,” GS1. [Online]. Elérhető: <https://www.gs1.org/standards/barcodes>. [Hozzáférés: 2026. május 14.]

[2] GS1, „EAN/UPC barcodes,” GS1. [Online]. Elérhető: <https://www.gs1.org/standards/barcodes/ean-upc>. [Hozzáférés: 2026. május 14.]

[3] GS1 US, „What is a GTIN?,” GS1 US. [Online]. Elérhető: <https://www.gs1us.org/upcs-barcodes-prefixes/what-is-a-gtin>. [Hozzáférés: 2026. május 14.]

[4] OpenCV, „Image Processing (imgproc module),” OpenCV Documentation. [Online]. Elérhető: https://docs.opencv.org/3.4/d7/da8/tutorial_table_of_content_imgproc.html. [Hozzáférés: 2026. május 14.]

[5] OpenCV-Python Tutorials, „Image Thresholding,” OpenCV-Python Tutorials. [Online]. Elérhető: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_thresholding/py_thresholding.html. [Hozzáférés: 2026. május 14.]

[6] OpenCV-Python Tutorials, „Morphological Transformations,” OpenCV-Python Tutorials. [Online]. Elérhető: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_morphological_ops/py_morphological_ops.html. [Hozzáférés: 2026. május 14.]

[7] S0dium, „DEAL KAIST Lab Barcode Main,” Kaggle. [Online]. Elérhető: <https://www.kaggle.com/datasets/s0dium/deal-kaist-lab-barcode-main/data>. [Hozzáférés: 2026. május 14.]

[8] „EAN-13 vonalkódolvasó Python implementáció,” saját fejlesztésű forráskód, 2026.

[9] „EAN-13 vonalkódolvasó statisztikai kiértékelése 1200 képen,” saját mérési eredmények, 2026.